# Smartbody en GitLab paso a paso

Etapa 5 · Lanzamiento con GitLab CI/CD (y lo que vino después: Docker, Kubernetes, Nginx, Prometheus/Grafana y microservicios)

Desde iniciar sesión con la cuenta de GitHub hasta publicar una versión con un clic de aprobación.

Proyecto académico · Ingeniería de Sistemas · Nicoll Flórez · octubre de 2026

# Qué vamos a recorrer

1. Crear la cuenta e iniciar sesión en GitLab con GitHub
2. Crear el proyecto y subir el código desde el PC
3. Activar los runners (quién ejecuta el pipeline)
4. El archivo `.gitlab-ci.yml` : cómo se define un pipeline
5. Primera ejecución: etapas, jobs, pestaña *Tests*
6. Repositorio de artefactos: Package Registry
7. Lanzar una versión: inputs, aprobación manual, tag y Release
8. Variables y tokens: dónde viven los secretos
9. Imágenes Docker: Container Registry con Kaniko
10. Trabajar con ramas y Merge Requests
11. Despliegue de prueba en Kubernetes (kind) y pruebas end-to-end
12. Errores reales que tuvimos y cómo se resolvieron
13. Mantenimiento: reintentos, `ci.skip` , limpieza

**Build**
compilar backend, Android y servicios

**Test**
JUnit: 85 + 22 + 88 pruebas

**Package**
jar + apk

**Docker**
5 imágenes con Kaniko

**Deploy**
kind + 36 aserciones e2e

**Publish / Release**
Package Registry, tag, Release

# 1. Iniciar sesión en GitLab con la cuenta de GitHub

1. Entra en **https://gitlab.com/users/sign_in** y pulsa **Sign in with GitHub** (o *Register* → *GitHub* si es la primera vez).

2. GitHub pide autorizar a GitLab: **Authorize GitLab**. GitLab crea la cuenta con el mismo usuario ( `Nicollflorez` ).

3. Completa el formulario de bienvenida (rol, uso) y confirma el correo si lo pide.

4. **Verificación de la cuenta**: para usar los runners gratuitos de gitlab.com hay que validar la identidad (teléfono o tarjeta, sin cobro). Sin esto, el primer pipeline queda en *failed* sin jobs.

> Lo vivimos: los dos primeros pipelines fallaron con "0 jobs" hasta validar la cuenta. Después, un pipeline nuevo ya creó sus 7 jobs.

# 2. Crear el proyecto

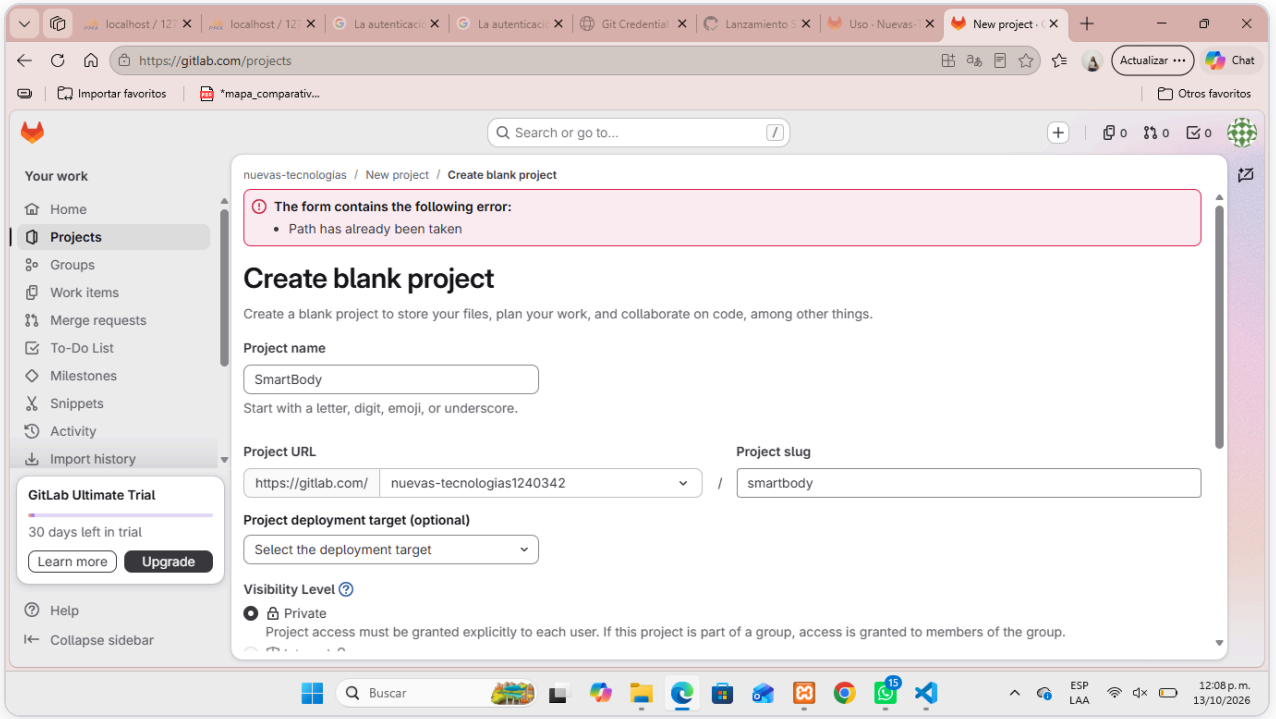
1. **+ → New project → Create blank project**.

2. **Project name:** `SmartBody` . **Project URL:** elige tu **usuario**, no un grupo privado: un proyecto no puede ser más visible que su grupo.

3. **Visibility:** *Public* (runners y registros sin límite; el profesor puede ver todo).

4. **Desmarca** "Initialize repository with a README": vamos a subir un repositorio que ya existe.

5. **Create project**. Resultado: `https://gitlab.com/Nicollflorez/smartbody`




> Si aparece "Path has already been taken" es que ya existe un proyecto con ese nombre en ese espacio: cámbialo de espacio (usuario) o de nombre.

# 3. Subir el código desde el PC

El proyecto ya estaba en Git (y en GitHub). Solo se agrega GitLab como segundo remoto:

```
# en la carpeta del proyecto
git remote add gitlab https://gitlab.com/Nicollflorez/smartbody.git
git push -u gitlab main
```

Al hacer `push` por HTTPS, el **Git Credential Manager** de Windows abre una ventana para iniciar sesión en GitLab (con GitHub). Se hace una sola vez.

> Si dejaste marcado "README", el push se rechaza (*non-fast-forward*). Solución sin forzar: `git merge gitlab/main --allow-unrelated-histories` y volver a hacer push.

## Detalles que importan en Windows

- `.gitattributes` fuerza **LF** en `mvnw`, `gradlew` y `*.sh`: los runners son Linux.
- El bit de ejecución no existe en Windows: `git update-index --chmod=+x backend/mvnw android/gradlew`. Sin esto el job falla con *Permission denied*.
- Nunca se suben claves ni contraseñas: `backend/keys/`, `*.pem`, `.env` están en `.gitignore`.

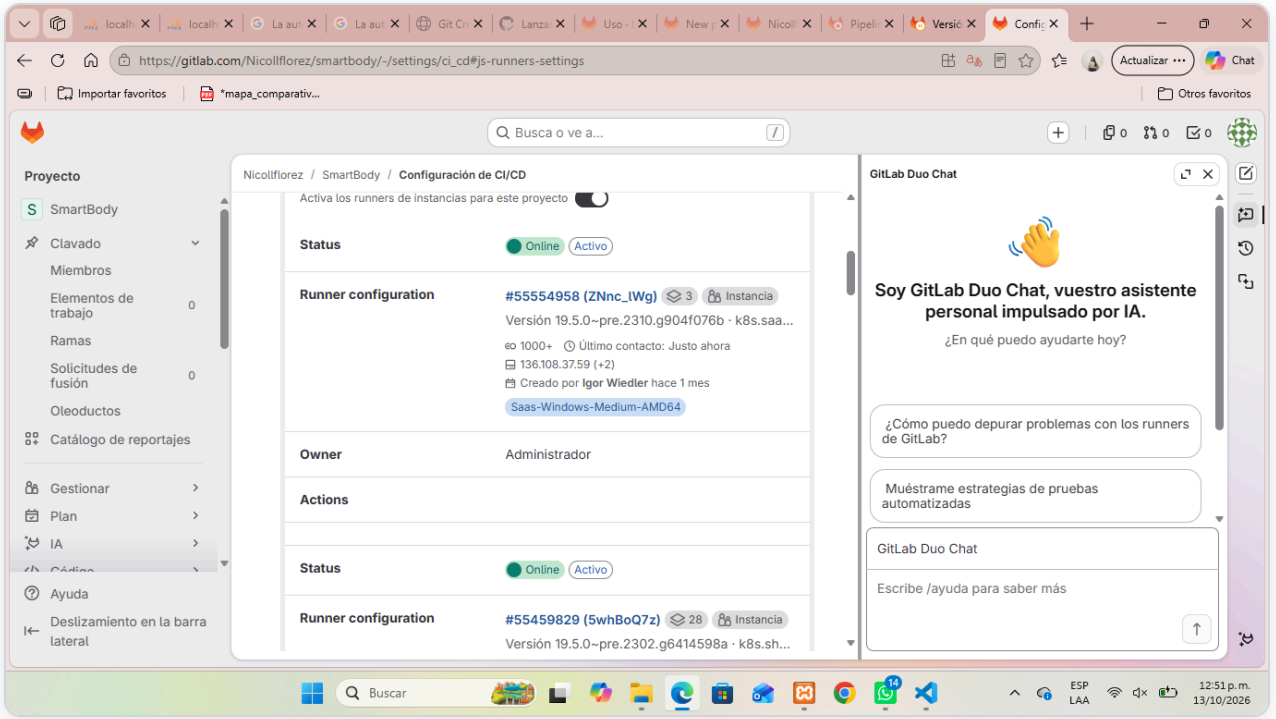
# 4. Runners: quién ejecuta el pipeline

Un **runner** es la máquina que corre cada job. gitlab.com ofrece **instance runners** compartidos (Linux, 2 vCPU) gratis para proyectos públicos.

1. **Settings → CI/CD → Runners**.

2. Interruptor **"Enable instance runners for this project"** activado; debajo se listan los runners *Online*.

3. Un job sin etiqueta (`tags`) cae en el runner Linux pequeño; algunos usan Docker-in-Docker (`services: docker:dind`) para crear contenedores dentro del job.




> La primera vez el job puede quedar *pending* 10-15 min: es cola de runners gratuitos, no un error (ver captura "Este trabajo aún no ha empezado").

# 5. El pipeline se define en `.gitlab-ci.yml` (raíz del repo)

```yaml
# Opciones del formulario "Run pipeline"
spec:
  inputs:
    publicar_release: { type: boolean, default: false }
    version_release:  { type: string,  default: "" }
---
stages: [version, build, test, package, docker, deploy, publish, release]

variables:
  PUBLICAR_RELEASE: "$[[ inputs.publicar_release ]]"
  MVN: "./mvnw -B -ntp"

workflow:            # cuándo se crea un pipeline
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH == "main"

build:backend:       # un job
  stage: build
  image: eclipse-temurin:17-jdk
  needs: [version]
  script:
    - cd backend
    - $MVN -DskipTests compile
```

## Conceptos

- **stage**: fase ordenada. **job**: unidad que corre en un runner.
- **image**: contenedor Docker donde corre el job (JDK, Android SDK, Kaniko, kubectl…).
- **script**: comandos; si uno falla, el job falla.
- **needs**: dependencias reales (DAG): el job arranca en cuanto sus *needs* terminan, sin esperar toda la fase.
- **artifacts**: archivos que pasan a jobs siguientes (jar, apk) y **reports: junit** alimenta la pestaña *Tests*.
- **cache**: `.m2` y `.gradle` para no descargar dependencias cada vez.
- **rules / when: manual**: condiciones y botón de aprobación.
- **spec:inputs**: campos del formulario *Run pipeline* (la interfaz nueva ya no muestra variables).

# 6. Las fases del pipeline de Smartbody

| Stage | Jobs | Qué hace |
|---|---|---|
| **version** | `version` | Lee la versión del `pom.xml` ( `1.0.x-SNAPSHOT` ); si es release, quita `-SNAPSHOT` y comprueba que el tag no exista. La publica como *dotenv* para todos los jobs. |
| **build** | `build:backend` , `build:android` , `build:servicios` | Compila con Maven (backend y 4 microservicios) y Gradle (APK release). En paralelo. |
| **test** | `test:backend` , `test:android` , `test:servicios` | JUnit: 85 + 22 + 88 pruebas. Reportes a la pestaña *Tests*. Si falla, no se publica nada. |
| **package** | `package:backend` , `package:servicios` | Empaqueta los jars (artefactos del pipeline). |
| **docker** | `docker:backend` , `docker:servicios` (matriz ×4) | Kaniko construye las imágenes **a partir del jar ya probado** y las sube al Container Registry. |
| **deploy** | `nginx:validate` , `k8s:validate` , `k8s:kind` , `k8s:kind:microservicios` | Valida Nginx y manifiestos; crea un clúster **kind** dentro del runner, despliega y ejecuta 21 peticiones end-to-end (Newman). |
| **publish** | `publish:snapshot` / `publish:release` (manual) | Sube jar y apk al **Package Registry**. En release espera el botón ▶. |
| **release** | `release` | Commit "Release vX", tag, bump a la siguiente versión y **Release** de GitLab con enlaces. |

# 7. Primera ejecución y cómo leerla

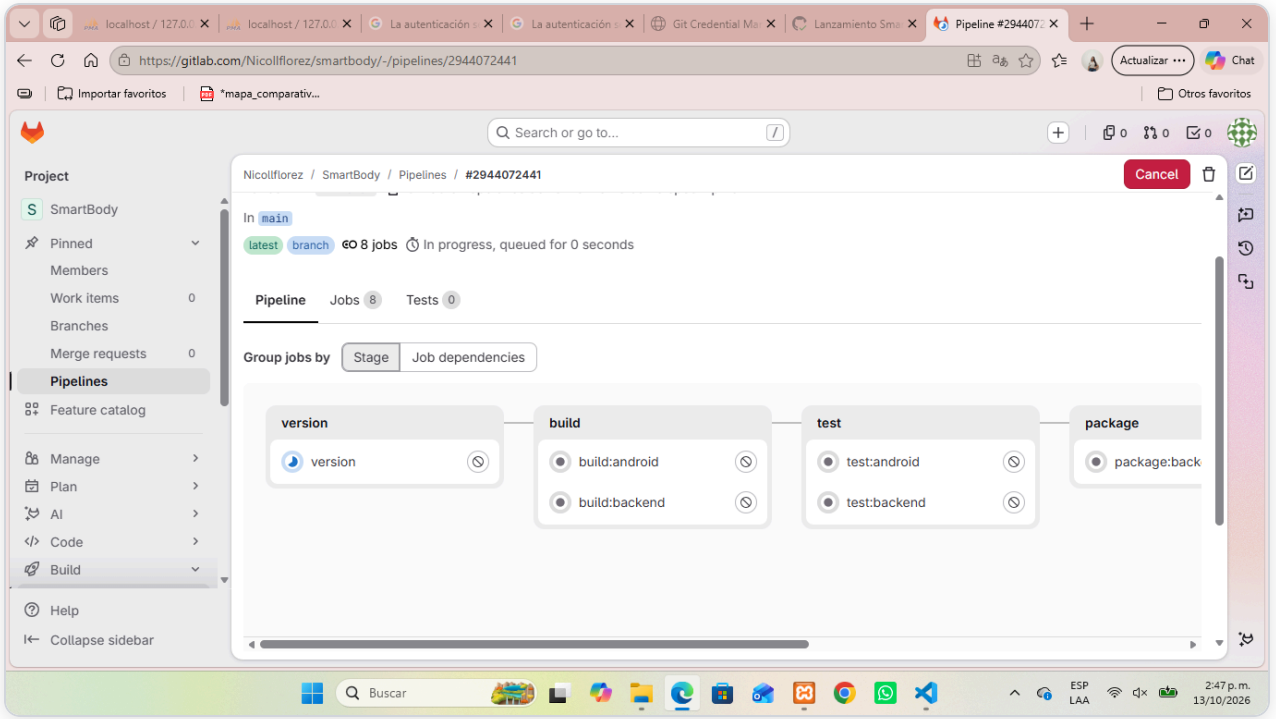
1. Cada `git push` a `main` crea un pipeline: **Build → Pipelines**.

2. Entra al pipeline: el grafo muestra las fases y el estado de cada job (azul: corriendo, verde: ok, rojo: fallo, gris: en espera).

3. Clic en un job → log completo. Al final: *Job succeeded* o la causa del fallo.

4. Pestaña **Tests**: suites JUnit con pasadas/fallidas por job.

5. Botón **Download** (artefactos) y **Retry** por job.




> Pipeline de referencia: #2946610215 — 22 jobs, 5 suites de pruebas, 5 imágenes publicadas.

# 8. Repositorio de artefactos: Package Registry

Cumple el rol de Nexus/Artifactory sin instalar nada: un repositorio Maven propio del proyecto. Cada compilación que pasa las pruebas deja el jar y el apk **versionados**.

```
# job publish (.gitlab-ci.yml)
$MVN -s ../devops/maven/settings.xml deploy:deploy-file
  -DrepositoryId=gitlab
  -Durl=$CI_API_V4_URL/projects/$CI_PROJECT_ID/packages/maven
  -DpomFile=pom.xml -Dfile=target/smartbody-backend-$VERSION.jar
```

```
# devops/maven/settings.xml: autenticación con el token del job
<server><id>gitlab</id>
  <configuration><httpHeaders><property>
    <name>Job-Token</name><value>${env.CI_JOB_TOKEN}</value>
  </property></httpHeaders></configuration>
</server>
```

## Dónde verlo

**Deploy → Package Registry**: `com/smartbody/smartbody-backend` y `smartbody-android`, versiones `1.0.x-SNAPSHOT` (desarrollo, se sobreescriben) y `1.0.1 … 1.0.4` (oficiales).

## Claves

- `CI_JOB_TOKEN` lo crea GitLab para cada job: no hay que guardar contraseñas.
- Se publica **el mismo jar que pasó las pruebas** (artefacto), nunca se recompila: promoción de artefactos.
- La versión vive en un solo sitio: `backend/pom.xml`; el pipeline la propaga al APK.

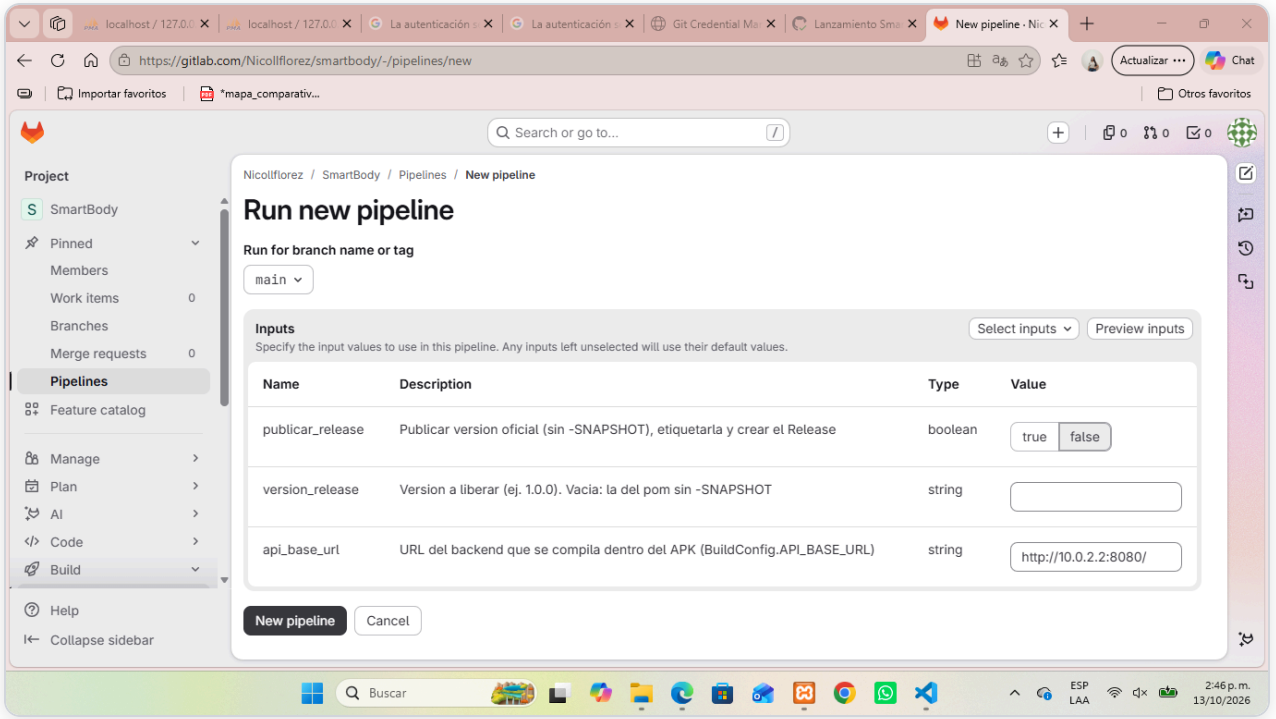
# 9. Lanzar una versión oficial (Run pipeline + aprobación)

1. **Build → Pipelines → New pipeline**, rama `main`.

2. En **Inputs**: `publicar_release` = **true**; `version_release` vacía (usa la del pom sin `-SNAPSHOT`) o una concreta; `api_base_url` = URL real del servidor para el APK.

3. **New pipeline**. Corre build, test, package, docker, deploy… y se **detiene en** `publish:release` con un botón ▶: esa es la aprobación del lanzamiento.

4. Al pulsar ▶: publica jar y apk oficiales; el job `release` crea el commit *Release vX.Y.Z*, el tag, el commit *X.Y.Z+1-SNAPSHOT* y el Release en **Deploy → Releases**.

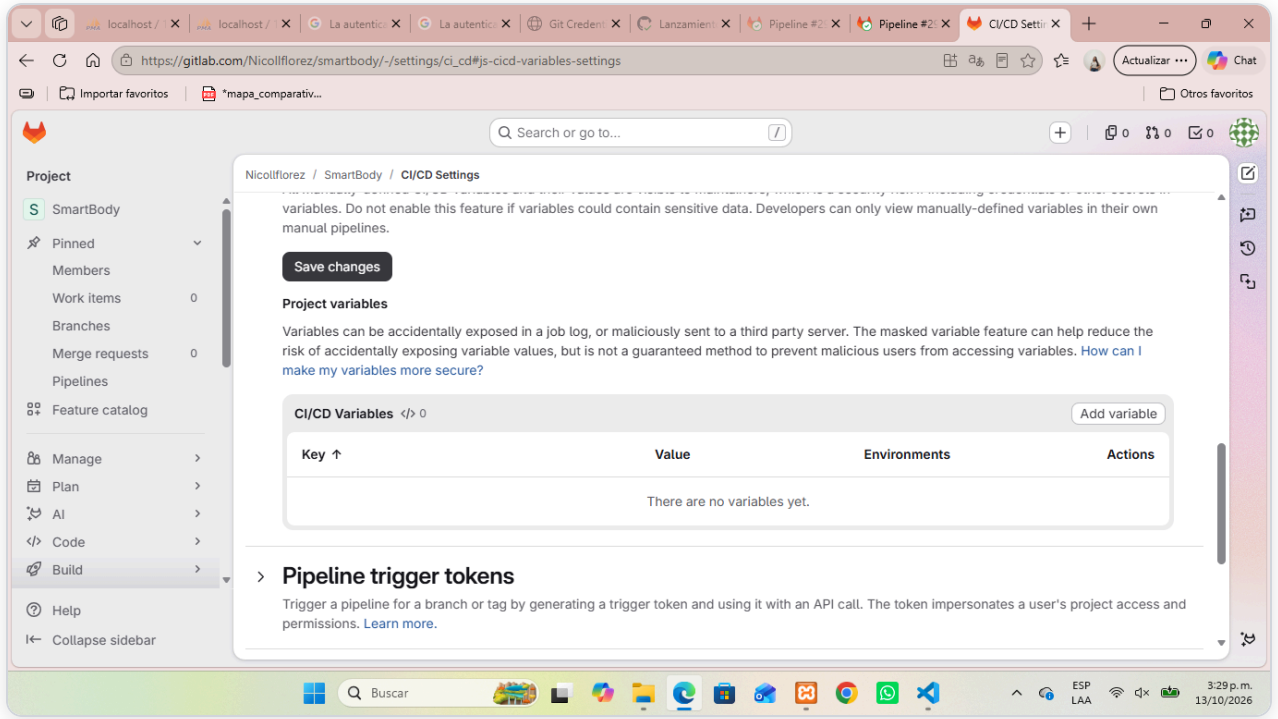


# 10. Variables de CI/CD: dónde viven los secretos

1. **Settings → CI/CD → Variables → Add variable**.

2. **Key** `GITLAB_PUSH_TOKEN`, **Value** el token, **Masked** (nunca aparece en logs), *Protect* desmarcado, *Environments: All*.

3. En el job se usa como variable de entorno: `$GITLAB_PUSH_TOKEN`. Si no existe, el job `release` salta el commit de versión (lo avisa en el log).




## Variables que GitLab ya da

- `CI_JOB_TOKEN`: Package/Container Registry.
- `CI_REGISTRY`, `CI_REGISTRY_USER`, `CI_REGISTRY_PASSWORD`, `CI_REGISTRY_IMAGE`: Kaniko.
- `CI_COMMIT_BRANCH`, `CI_COMMIT_SHA`, `CI_PIPELINE_IID`, `CI_PROJECT_ID`, `CI_API_V4_URL`.

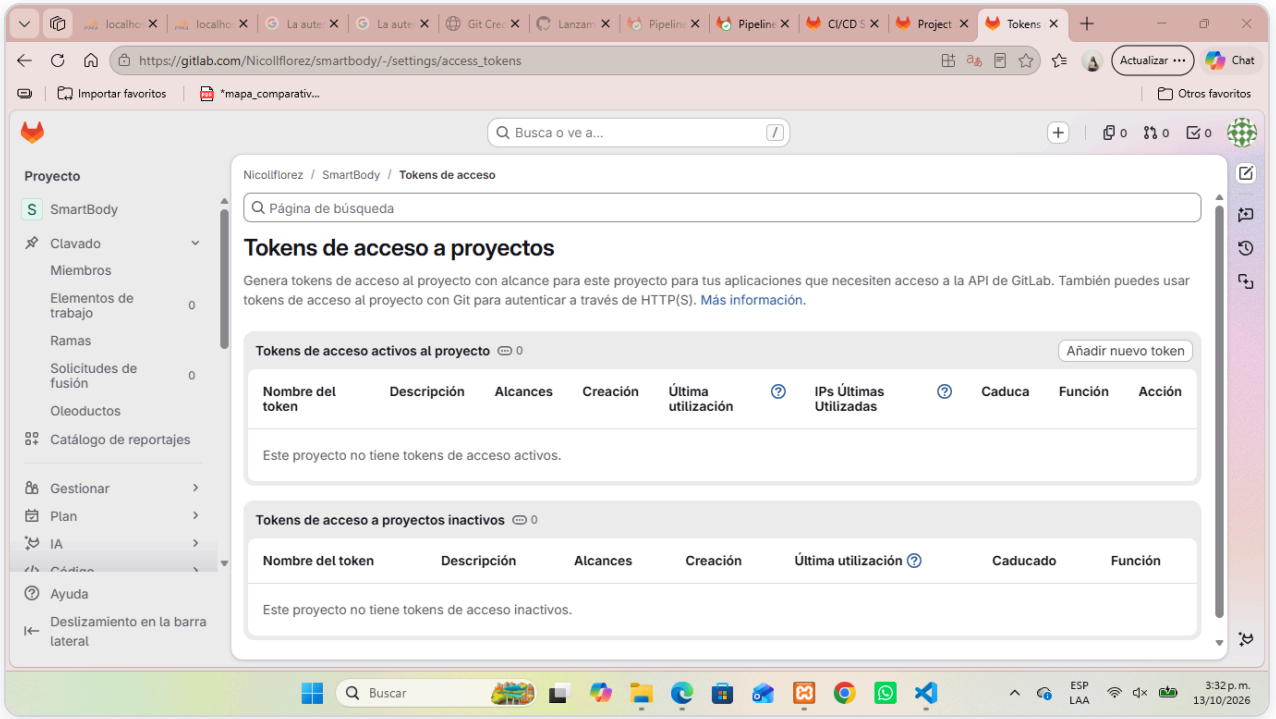
> Un valor pegado en un chat, un log o un commit ya no es secreto: revócalo y crea otro.

# 11. Tokens de acceso al proyecto

Un **project access token** es una "cuenta de servicio" del proyecto. Se crea en **Settings → Access tokens → Add new token**.

| Para qué | Rol | Scope |
|---|---|---|
| Que el job `release` haga commit y push a `main` | Maintainer | `write_repository` |
| Borrar pipelines, crear Releases por API | Maintainer | `api` |
| Solo leer (consultas, paneles) | Reporter | `read_api` |

- El valor ( `glpat-…` ) se muestra **una sola vez**: cópialo al crearlo.
- Guárdalo como variable **Masked** de CI/CD o úsalo desde tu terminal; nunca en el código.
- Ponle caducidad corta y revócalo cuando termine su uso.



# 12. Imágenes Docker: Container Registry con Kaniko

```yaml
docker:servicios:
  stage: docker
  image: { name: gcr.io/kaniko-project/executor:v1.23.2-debug, entrypoint: [""]
  needs: [version, package:servicios]
  retry: 2
  parallel:
    matrix:
      - SERVICIO: [auth, planes, progreso, admin]
  script:
    - echo "{\"auths\":{\"$CI_REGISTRY\":{\"auth\":\"$(printf %s:%s $CI_REGISTR
    - /kaniko/executor --context $CI_PROJECT_DIR/services
        --dockerfile $CI_PROJECT_DIR/services/Dockerfile
        --build-arg SERVICIO=$SERVICIO --build-arg VERSION=$VERSION
        --destination $CI_REGISTRY_IMAGE/$SERVICIO:$VERSION
```

- **Kaniko** construye imágenes sin demonio Docker (los runners compartidos no lo permiten directamente).
- **parallel: matrix** crea 4 jobs a la vez, uno por servicio.
- **retry: 2** repite el job si falla por algo transitorio (red, registro).
- Las imágenes quedan en **Deploy → Container Registry**: `backend`, `auth`, `planes`, `progreso`, `admin`, con tags `1.0.x-SNAPSHOT`, `1.0.4` y `latest` (solo desde `main`).
- Se construyen **desde el jar probado** (artefacto), no desde el código.

# 13. Ramas y Merge Requests

Los cambios grandes (Etapas 6-8, pruebas e2e, microservicios) se hicieron en ramas para no tocar `main` hasta tener el pipeline en verde.

```
git checkout -b etapa-6-7-8
# ... cambios y commits ...
git push -u gitlab etapa-6-7-8 \
  -o merge_request.create \
  -o merge_request.target=main \
  -o merge_request.title="Etapas 6 y 7"
```

- Las **push options** crean el Merge Request desde la terminal.
- El MR tiene su propio pipeline (`merge_request_event`): valida antes de integrar.
- Al estar en verde: **Merge** (o `git merge --ff-only` y push a `main`).

## Lo que hicimos

| MR | Contenido | Resultado |
|---|---|---|
| !1 | Docker, Kubernetes, Nginx, Prometheus/Grafana | 3 iteraciones (YAML, kind con dind) → verde |
| !2 | Pruebas end-to-end de la API (Postman/Newman) | verde a la primera |
| !3 | 4 microservicios + gateway | 3 iteraciones (Kaniko transitorio, CPU de MySQL) → verde |

> Regla: nada entra a `main` con el pipeline en rojo.

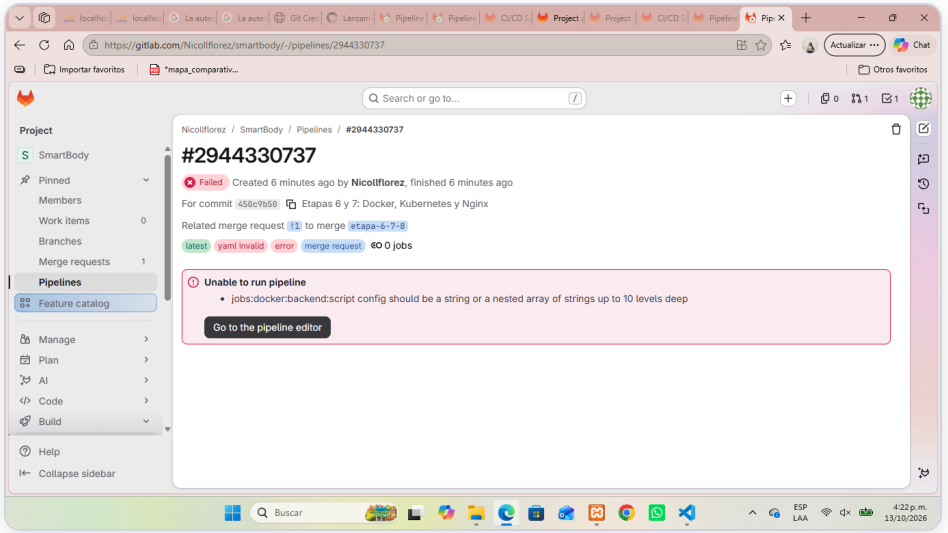
# 14. Despliegue de prueba en Kubernetes dentro del pipeline

```
k8s:kind:
  image: docker:27
  services: [docker:27-dind]          # Docker-in-Docker
  script:
    - kind create cluster --name smartbody --config k8s/ci/kind.yaml
    - kubectl config set-cluster kind-smartbody --server=https://docker:$PUERTO
    - BACKEND_IMAGE=$CI_REGISTRY_IMAGE/backend:$VERSION sh k8s/desplegar.sh
    - curl -sf http://docker:30080/api/ping
    - newman run tests/e2e/smartbody.postman_collection.json
        --env-var baseUrl=http://docker:30080 --reporters cli,junit
  after_script:                           # diagnóstico aunque falle
    - kubectl -n smartbody get pods -o wide
    - kubectl -n smartbody get events | tail -25
    - kind delete cluster --name smartbody
```

- **kind** crea un clúster Kubernetes real dentro del runner en ~2 min.
- Se despliegan MySQL, backend (o los 4 servicios), Nginx, Prometheus y Grafana con los manifiestos de `k8s/`.
- **Newman** ejecuta la colección Postman: 21 peticiones, 36 aserciones, reporte en *Tests*.
- Mismo criterio para monolito y microservicios: **la misma colección debe pasar en ambos**.
- El clúster se destruye al terminar: cada ejecución parte de cero.

# 15. Errores reales y cómo se diagnosticaron

| Síntoma | Causa | Solución |
|---|---|---|
| Pipeline *failed* con 0 jobs | Cuenta sin validar | Validar la cuenta; relanzar |
| "yaml invalid: script should be a string" | Una línea con `:` (dos puntos + espacio) se leía como mapa | Entrecomillar la línea |
| *Permission denied* en `mvnw` | Sin bit de ejecución desde Windows | `git update-index --chmod=+x` |
| Job `release` falla sin motivo claro | Sintaxis de bash ( `<<<` ) en imagen Alpine ( `sh` ) | Sintaxis POSIX |
| Etiqueta `v1.0.2` ya existe | Se relanzó sin cambiar versión | Comprobación al inicio del job `version` |
| `kind` : "timed out waiting for control plane" | API server en localhost del dind | `apiServerAddress: 0.0.0.0` , sin `--wait` |
| MySQL `Pending` 5 min | Suma de *requests* de CPU > 2 vCPU | Bajar *requests* a 50-100m |
| Release: "Links have duplicate..." | Dos enlaces con la misma URL | URLs únicas por imagen |




El mensaje de error del pipeline dice exactamente qué job y qué clave están mal.

# 16. Mantenimiento diario del pipeline

## Evitar ejecuciones inútiles

Un cambio solo de documentación no necesita 25 minutos de pipeline:

```
git push -o ci.skip gitlab main
```

GitLab registra un pipeline *Skipped* (gris): no es un error.

## Reintentar

- Botón **Retry** en el job (misma configuración).
- `retry: 2` en jobs con fallos transitorios (Kaniko).
- Un pipeline nuevo si cambió `.gitlab-ci.yml`: el retry usa la versión vieja.

## Limpiar el historial

- Abrir el pipeline → icono de **papelera** → *Delete pipeline*.
- Por API con un token `api`: `devops/gitlab/limpiar-pipelines.sh` borra *failed/skipped/canceled* y conserva los verdes.

## Leer un fallo rápido

1. Pipeline rojo → job rojo → final del log.
2. Buscar *ERROR*, *Caused by*, *exit code*.
3. En kind: el `after_script` ya imprime pods, eventos y logs.

> Hoy: 11 pipelines, todos en verde.

# 17. Resultado: del push al lanzamiento




## Entregables en GitLab

- Proyecto público: **gitlab.com/Nicollflorez/smartbody**
- Pipeline de referencia: **#2946610215** (22 jobs en verde)
- Releases: v1.0.1, v1.0.2, v1.0.3 (monolito) y v1.0.4 (con microservicios)
- Package Registry: `smartbody-backend`, `smartbody-android`
- Container Registry: `backend`, `auth`, `planes`, `progreso`, `admin`

## Documentación en el repositorio

- `docs/GESTION_DE_LANZAMIENTO.md` — Etapa 5
- `docs/DESPLIEGUE_Y_OPERACION.md` — Etapas 6, 7 y 8
- `docs/ARQUITECTURA_MICROSERVICIOS.md`
- `docs/PLANIFICACION_TRELLO.md` — Etapa 1
- `tests/e2e/` — colección Postman
- `.gitlab-ci.yml`, `Jenkinsfile`, `.github/workflows/`

# Gracias

Smartbody · ciclo de vida del software con herramientas open source

GitLab CI/CD · Docker · Kubernetes · Nginx · Prometheus · Grafana · Postman/Newman · Kaniko · kind

Repositorio: gitlab.com/Nicollflorez/smartbody · github.com/Nicollflorez/SmartBody1